BFUA-STUDY BUDDY-whatever you want to name it..

Core idea

A collaborative platform for university students to grow and professors to inspire and help.

This website will be designed for those lost students who cant access or find the answers for that one very specific question we all had atleast once in our academic journey. It's for when the student/researcher gets failed by all their used-to-go platforms(youtube ,AI, git-hub, stack-overflow…..),in short it will be a knowledge hub for peer to peer support and professor guidance , where users solve each other's problems and possibly learn new things in the process.

It might also serve as a support place and a showcase area where users share tips ,ideas ,plans of progress , road maps , or resources they find interesting -only in their designated areas-…..i lost my words there…….

What I realized in university is most of other majors don't get the support our major get(even if not much related)….we get to have the AI and git-hub and stack overflow….as limited as they are ,and as inaccurate as they can get, we still get the privilege most don't.

This platform might just solve that, because even if this idea was done before..its not very popular for other majors .

So we make it as simple for the non thec-savvy people and as functional to be the very much help needed for all majors.. cause lets face it, the AI isnt good for everything

What we can attempt to make in short time and try the best to prefect is the first base of communication, try and make it Q&A functional at best we can-with photo exchange support-

First base of communication should make sure the user can post their questions/ideas as posts -again ,only in designated areas-,while the response from other users would be as comments, these posts/comments will be judged by others with the vote method.

The vote would be 2 or 3 types for the comments:

Useful :the regular up vote for when those who read the comment find as helpful for op(Original Poster).

Useless : for when the comment is offensive ,off -topic, or mockery …..basically something that is simply useless.

Specialized : the higher form of useful and is given by the special members of the community or when the op themselves find that comment exactly what they were hoping to find.

The usefull/useless votes will directly effect the user's account and ability of interacting with others on the platform through the ` rating system `….

The rating system: this is the counter of how many votes the user maintains through their posts/comments ….this might be just a simple counter display or will implement the 5 star rating type….the higher the useful counter the higher the rating gets and with it the more credibility that user will have thus others would find it easy to rely on their responses.. reaching a point of high(if we go with the 5 star type of rating ,we make it around 4stars),where is the users with that level reach ,comments on something, they will automatically get the "specialized " vote on their comment, they will also be able to give that vote to whom they think they deserve .

To keep the things balanced, each useless vote will deduct from the rating ,reducing the credibility of the user's comments and all……

Other then the high credibility users, op can only give specialized vote to the comments of their own posts,non more.

The post will simply obtain the useful /useless system…the higher the useful rate the more it will appear on other people's feed, the higher the useless rate, the lower the chance that post will appear .

One of the things I talked about before is the "designated areas".

What I mean by that is a special set- that I still to this moment not sure how to implement – of rooms.

5 main types of rooms to be precise, university room, field/major room, subject room, public space room, and the private rooms.

Self-explanatory really, each *university* will git its own *special room*,

Within each university there are majors, there for ..*major oriented room*…

For each major there are a set of *subjects* to study throughout the academic years…each with *their own room*..

The *public space room* is the free zone where the user can express ideas, share tips, or just be free with their thoughts and by that freedom be judged by other users through the rating system.

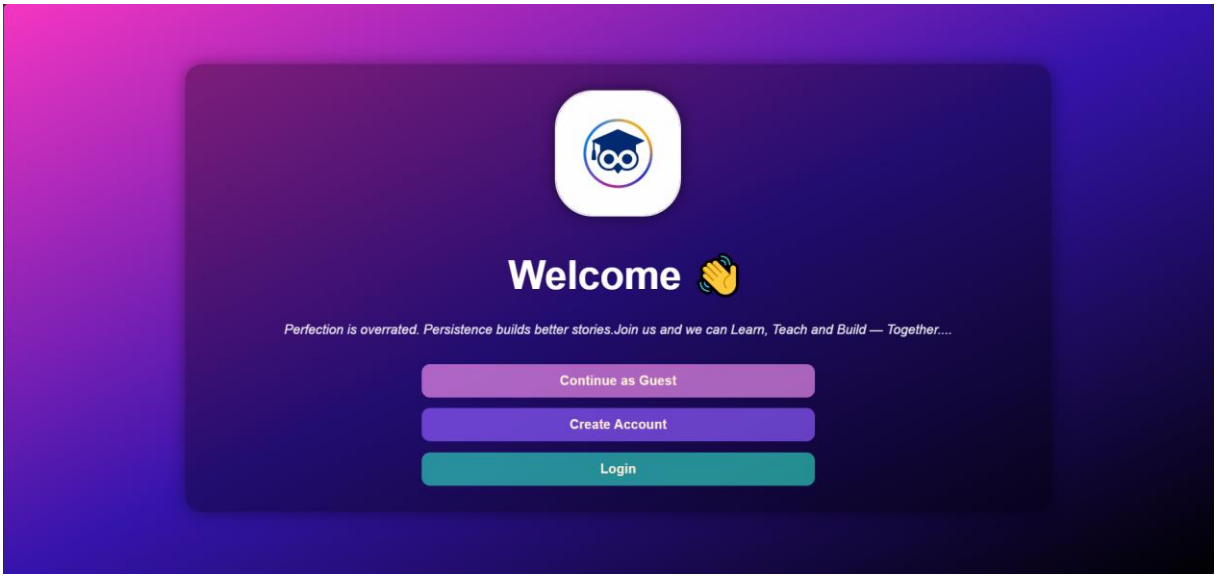
The *private rooms* are the by name suggests…private within the user who created it and who the user wants there…-this might work with creating a passcode as google classroom does, or a user created password-

That in my mind will give the users a clear specific oriented space to express their desires

Interfaces existing

Since my brain works backwords . i already framed the user interface and what the pages(most)would look like, sadly without using any framework ,we relied on only vanilla html ,css ,and JavaScript….and to imitate real app behavior (at most),used the local storage as our initial database…that is the first steps to insure the user interfaces are working as they should when passing data within each other…..(one of the most accruing problems I am facing is the names of the pages, will fix that eventually)

1-welcome.html./home.html(same page ,name problem)



Another problem of mine is my love for dark themes and interfaces ,although I tried to keep it out of my coding habits ,they still shine through (ps,I don't see this as dark ,neither half the people I asked ,the other half thinks its dark).

Back to the page ,this is the first interface the user will see when visiting the website, quite a simple one from the logo of the project, to the waving hand(yes, it waves)to our project's moto, to the 3 simple hard to miss buttons ….

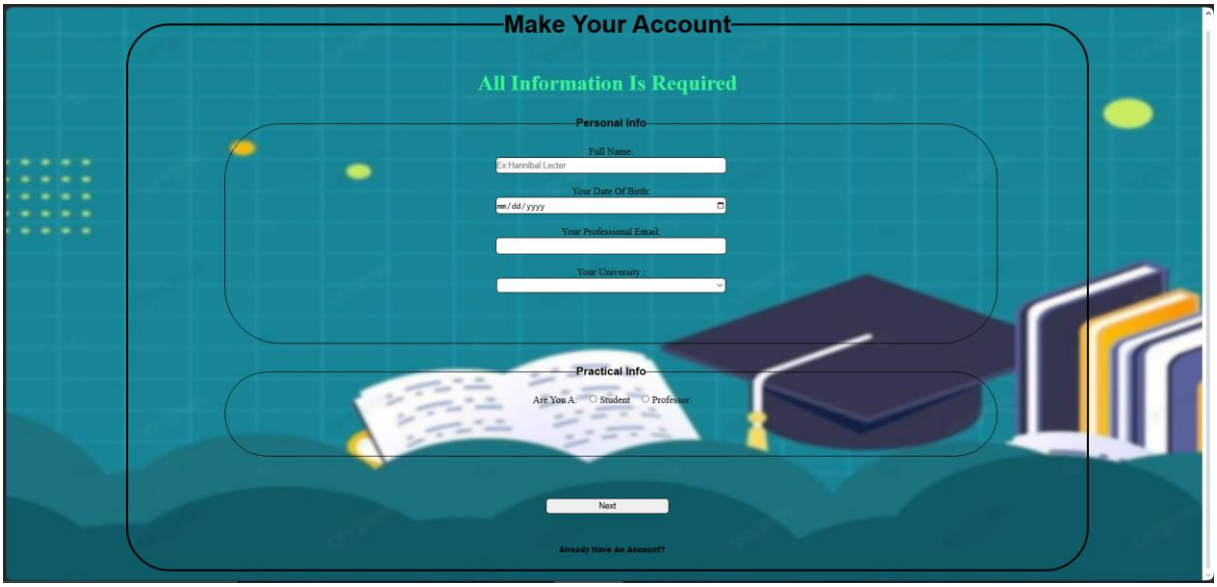
As for the buttons ,2 of them have a clear as day functions ,otherwise we will visit it later.

The golden button (might actually change its color to gold in that case) is the "continue as a guest" button, this is for the ones who are not sure what to expect from the platform so they can visit it as the unknown individuals they want to be, check the website then decide whether to join the family or find their fit elsewhere, with no strings attached.

By that, based on the designated area concept …. the guests can only visit and interact with the university room of their choice (multiple maybe) and the public space room.

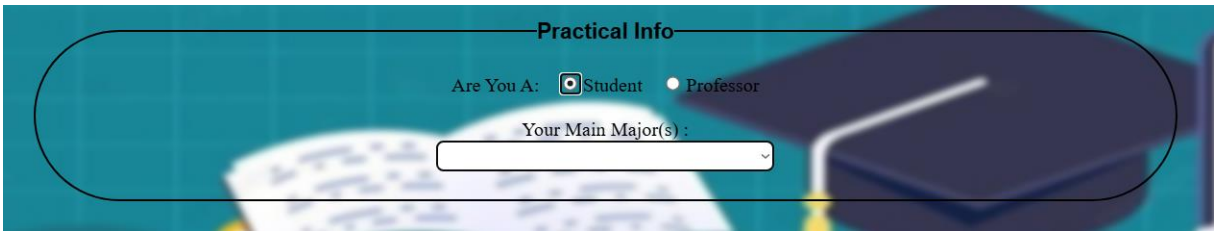
Moving on, the 'create account 'button would not surprisingly lead the user to the first step of creating their account for the platform.

2-Information form(info.html)



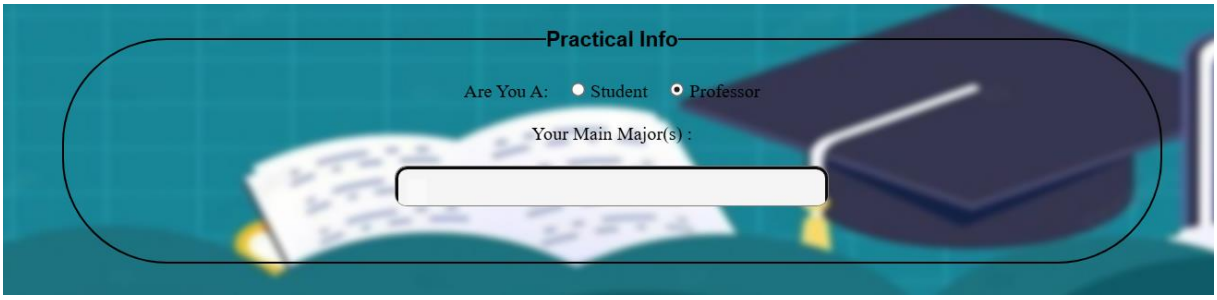
what looks better with auto overflow for better clarity in the browser rather than this screenshot, is this basic personal information form, where the user is required to fill information from their name, date of birth, email for security reasons, their university (for the room).to their role as a student or a professor then their major..

for students



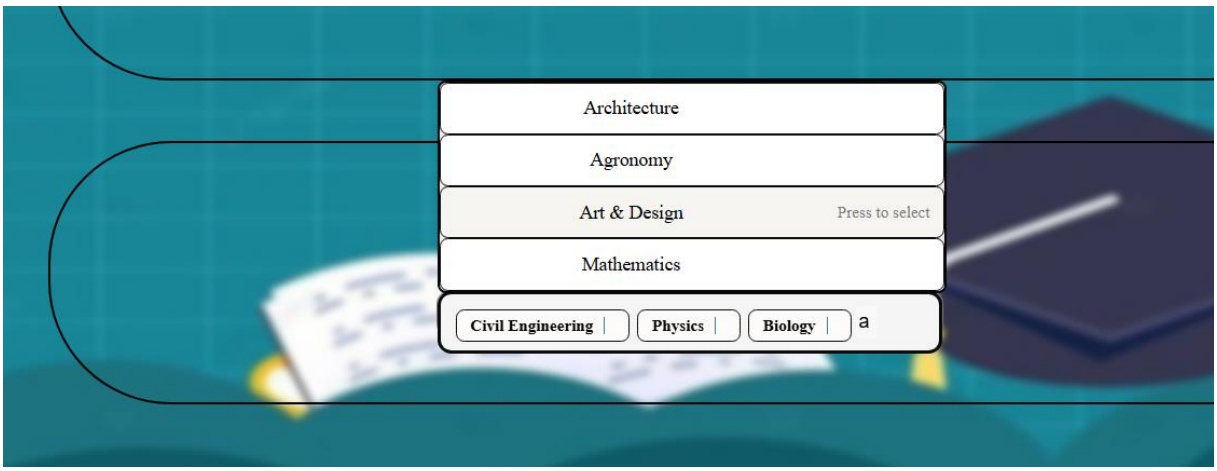
It's quite uncomplicated for each student would get to choose one specific major they are oriented in. thus this will add them automatically to the room of their major and the rooms of subjects related to that major.

For professors

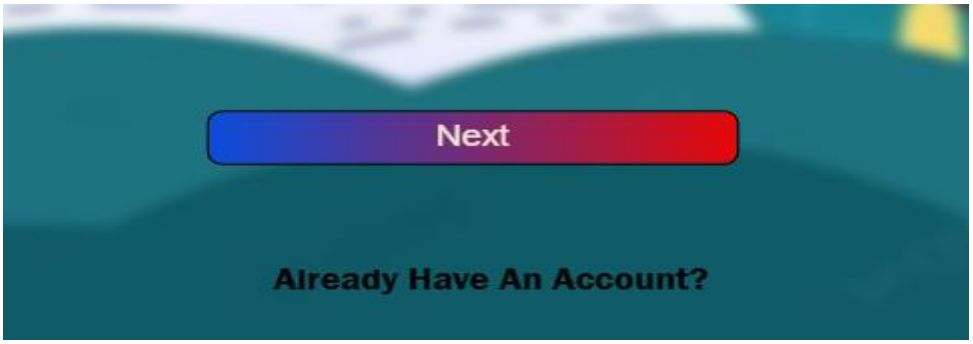


When it comes to professors, they can be tied to multiple majors rather than only 1, to fix that choice problem, we took the liberty to use a library called choices.js

This library as hard to style and design to be , is quite usefull in this context from the multiple selection, to the search and all…with that come the possibility of relying on it instead of the usual select…



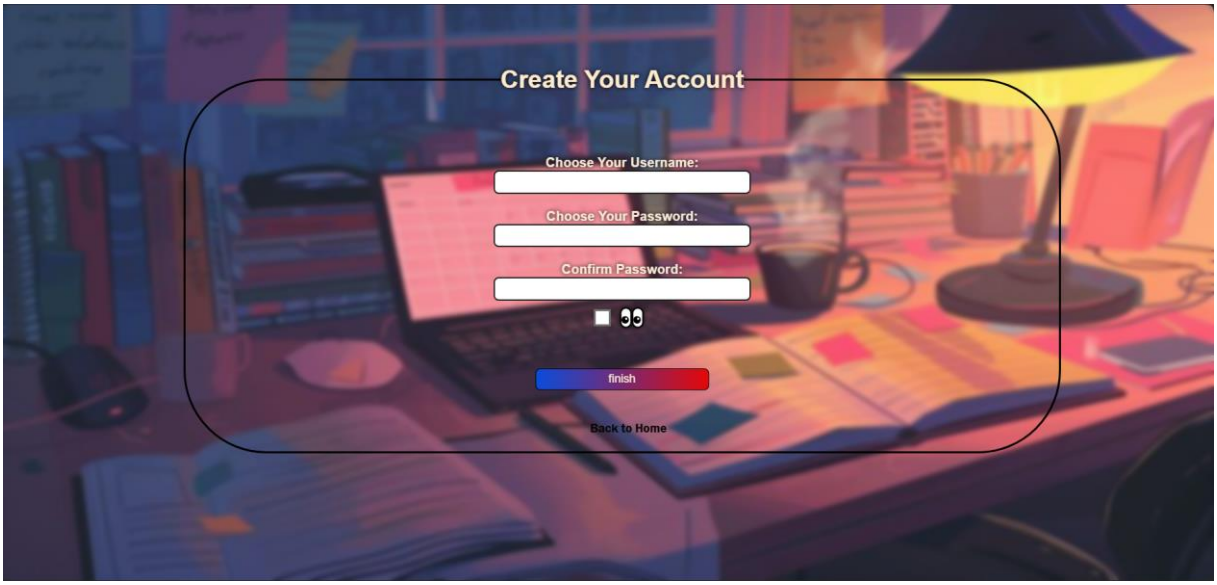
Another note on this page, as noticed at the end of it there are 2 options,



Simple button to when the user finished the form and a redirection link we'll talk about later.

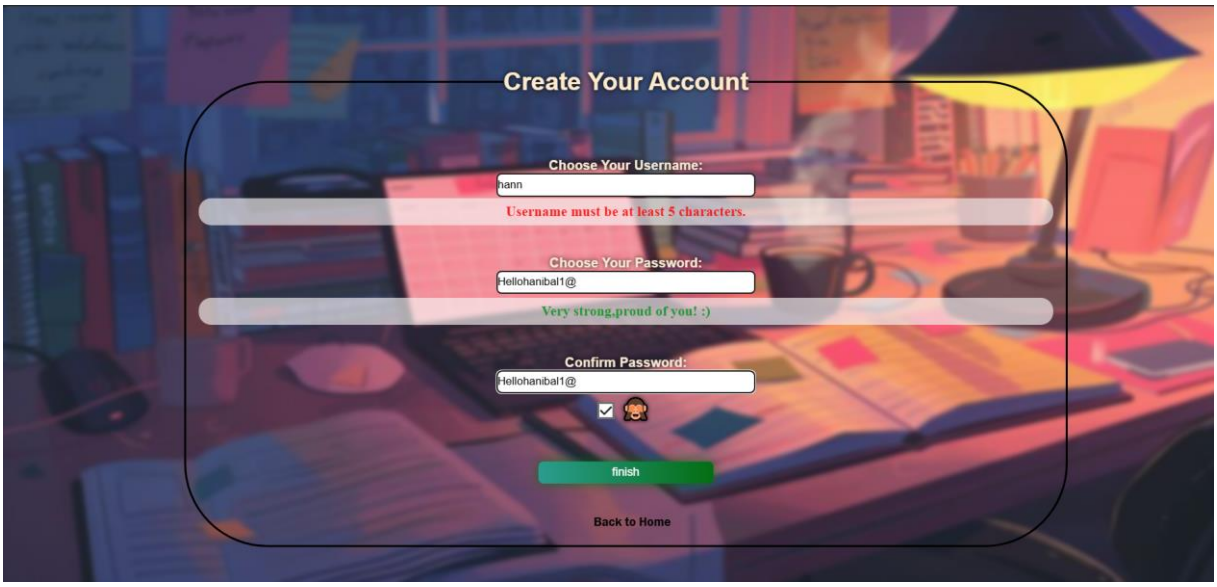
The 'next' button will take the user to their next step.

3-Register.html

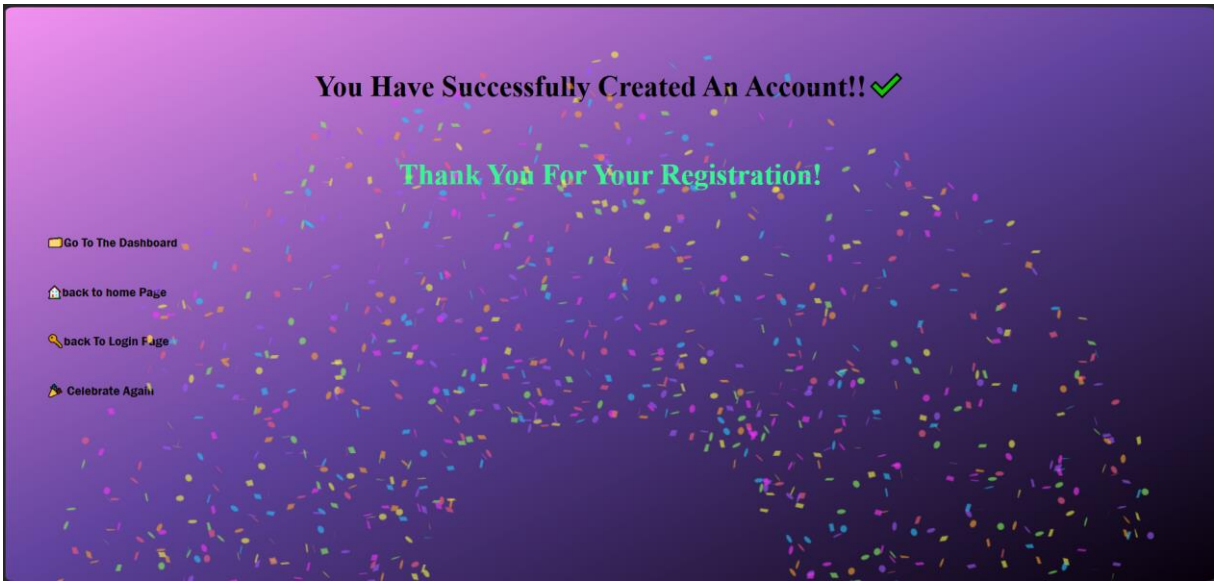


A hint of coziness in this page and a touch of playful vibes, this page is where the user chooses how they will display themselves in the platform, and how they will secure their accounts and privacy. yet this page has some regulations as not anything could pass, we used JavaScript to give in real time feed back on the strength of the password, how the user knows what they typed through “confirm password” a simple toggle to see is. And a set of regulations when it comes to the username the user will choose for themselves.

With the finish button only activated when the password types are matching the “confirm password “



4-finish.html



The celebration page, with a burst of confetti to joyfully welcome the new comers

That was its purpose, just a page to thank the user for joining the community.

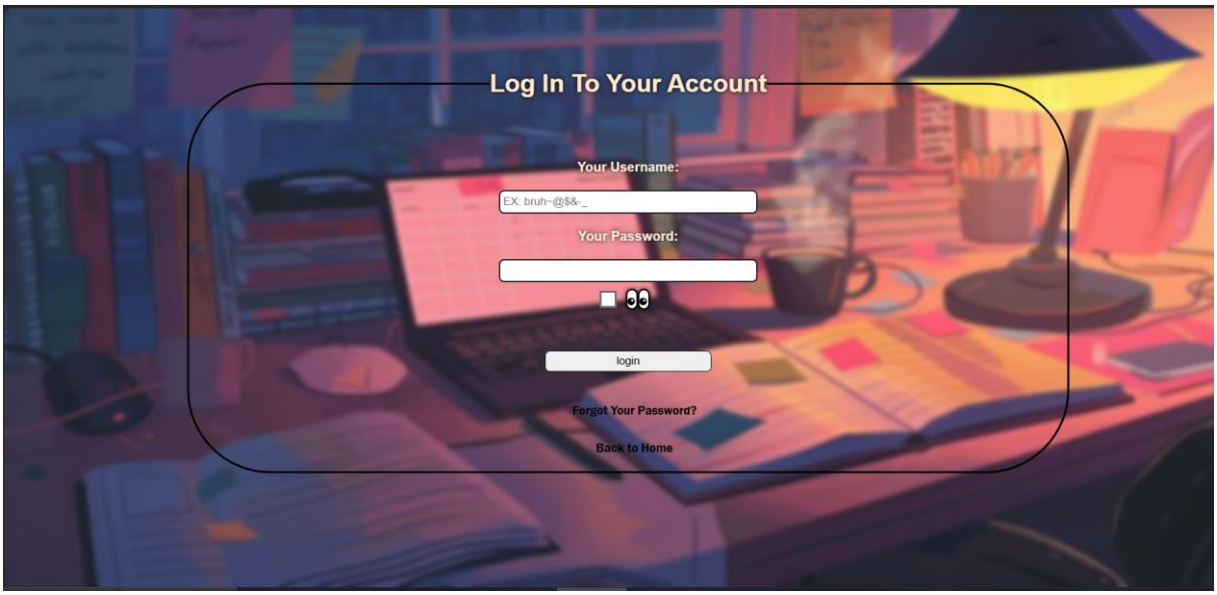
Then redirect them to their next step whatever they would feel like doing…

Or just click the confetti and enjoy the burst of colors.

Throw back to the third button on the first page the “login” button, and the “already have an account” in the second page….

Users who already went through the process of creating an account, although it will be fun to reach the “finish “page again, can’t deny filling all that info out again would be annoying + some users would rather have just 1 single account... there for we got:

5-login.html



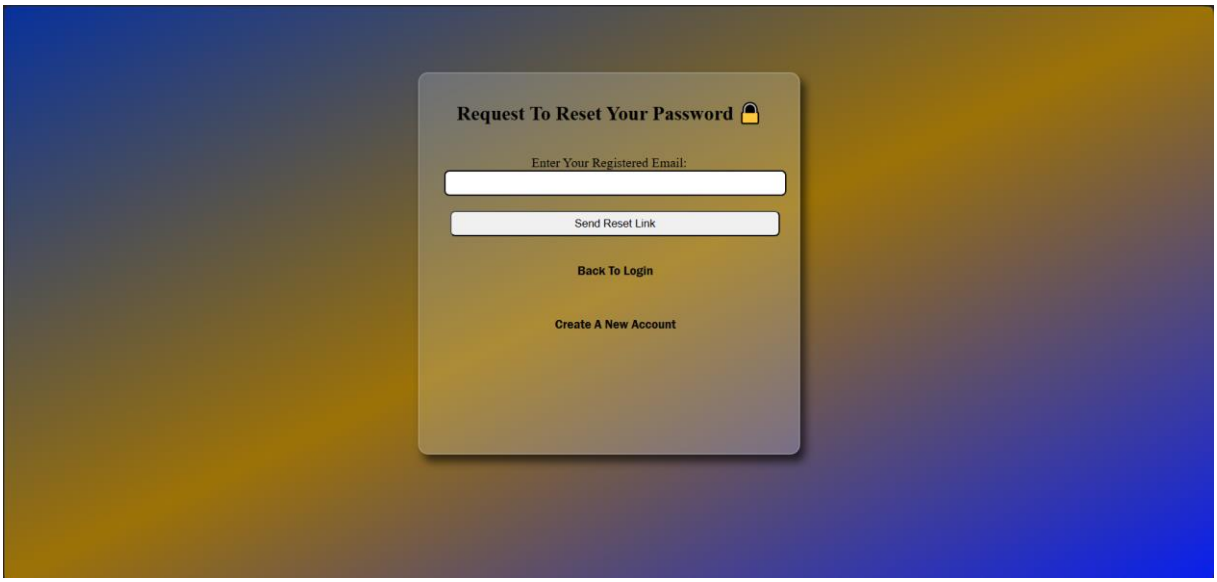
Maintaining the same layout as the register page, we tried to keep this page as simple as it gets.

Only checking is the data giving by the user actually exist in the database (in this case or level of production ,in the local storage) .if so ,the user will access their account and thus continue to the platform from there, else simply throw error messages of username not existing or password incorrect…

Speaking of incorrect passwords….we all forgot atleast one password in our digital life span.

Thus to deal with this, the “forgot your password?” link.

6-Request-to-reset-password

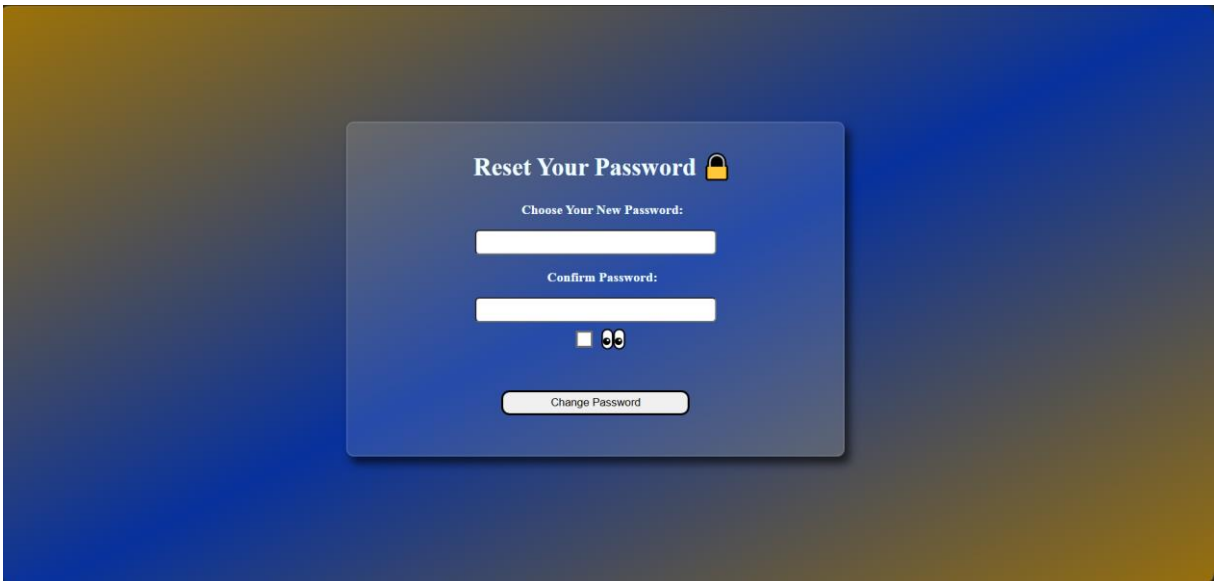


Using the EMAIL *given in the info from* during the early stages of creating an account that the databse will check its existence, users can request to change their password. Or just give up on the old account and go ahead with creating a new account from scratch.

This step is yet a work on progress with the plans of backend and the database.

Considering the email exists in the database, the app will send a link to the said email to reset the password through the said email.

7-reset password



the process is simple, change the password -same password confirmed- and this will directly update the database ,so next time ,the login will be with the new password (unless, at the time…that will also be forgotten).

Right that this point is where I realize one of the mistakes in this…

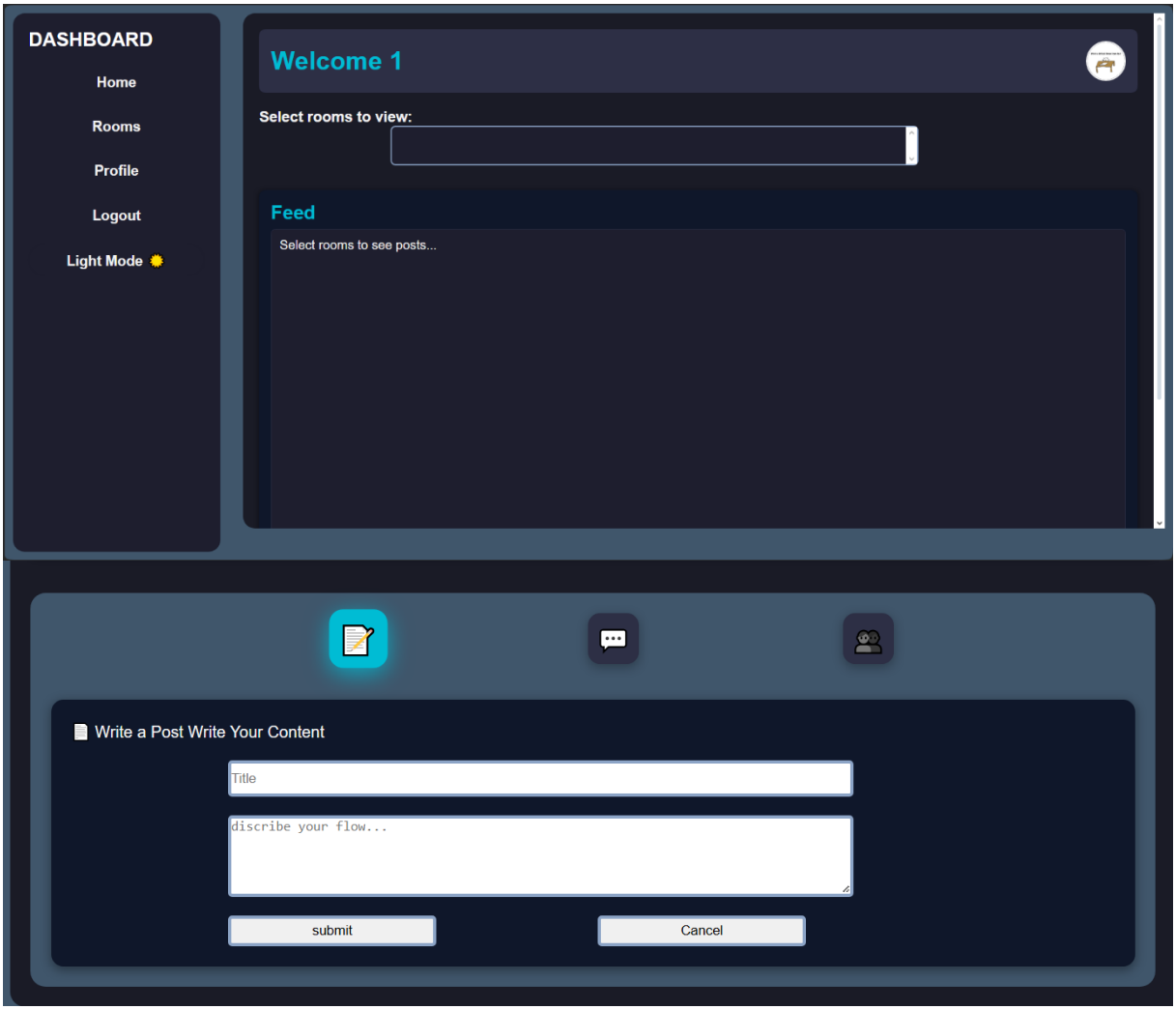
The login is only by username and password, yet recovering the account, users can only reset the password….. what if the user don't remember their username? that would mean the account is lost?

By logic, this fix would be rather simple, I'll add the email option to login with.

User can forget the username and password, that will be ok, but forgetting the registered email is tad too much, thus that account would be useless, this is added to the list of things that should be done.

Moving on to the next segment, where all the action and adventure appears to happen.

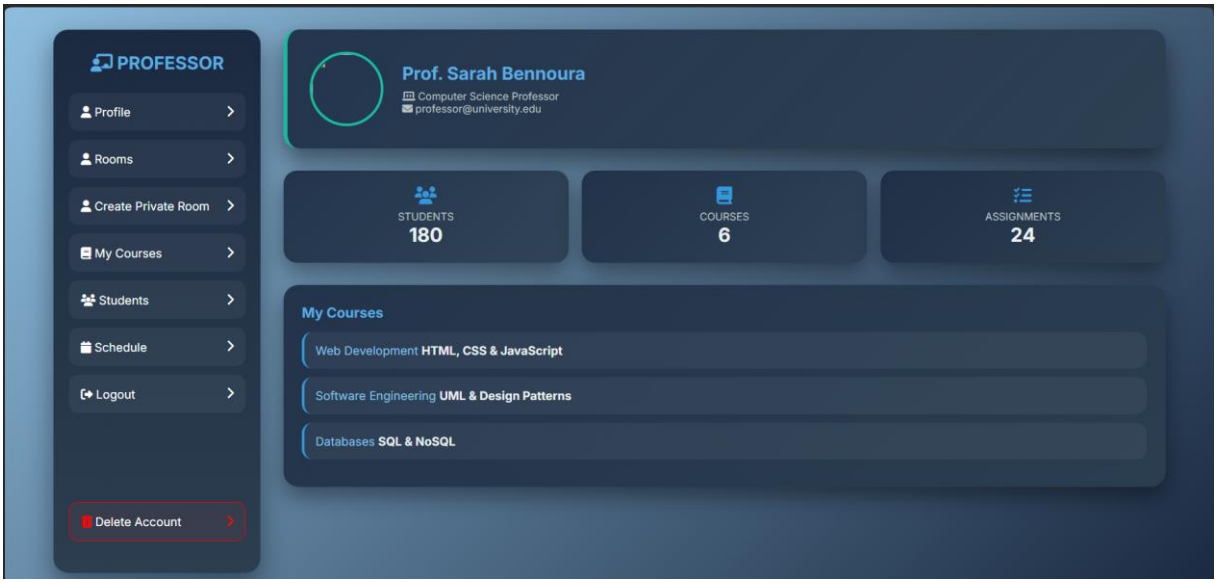
8-dashboard.html

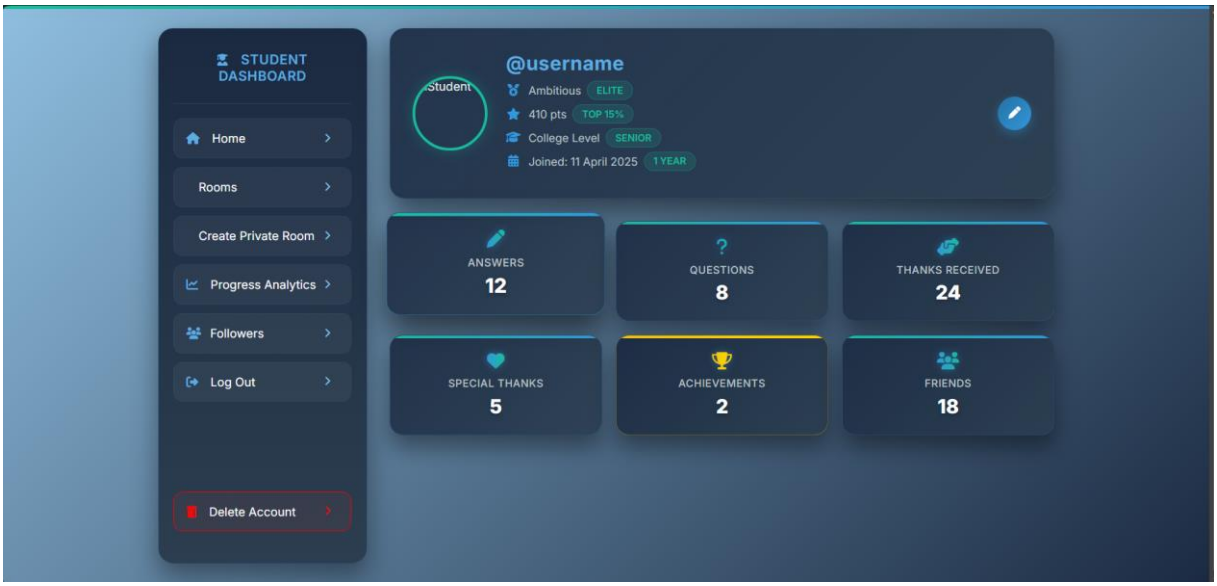


The dashboard so far has the heaviest duty and thus the most problematic build for this project, this will be the user's costume space where they will be spending most of their time…browsing, asking, seeking, answering, chatting, and working together .

It is this page that is the heart of the project, this should implement the rooms, with filter to have the freedom to browse what ever room the user feels like vising without being overwhelmed…

9-Profiles





sleek, modern and simple design to display the user's info based on their registered role given in the info form from the early stages,

this like the dashboard is still a work in progress, this still needs to be refined with a similer layout yet distinctive enough ….with a direct connection to the dashboard with live, in real-time status update.

What is and what should be done

Front-end(JavaScript +frameworks for the future)

As mentioned before, the work that is already done for is with vanilla css+js…no actual framework implementation yet…this is due to me lacking the better sense before I got one

step too deep and due to the lack of knowledge and the desire to test where would what I knew before lead me …..

Moving on, this segment is to site what I have done and what is yet to be done….in hopes that we will be able to use the frameworks to ease what is left.

I will be pasting lists of my initial thoughts on what JavaScript should have directly from my scripts in the project to ease out the writing, there for there will be some odd expressions here and there… much like all my other writings so far.

1-welcome page

Thai page is to redirect users to where they need to go….

Quite simple and little to no work for 2/3 of the buttons….

The biggest work is the guests logic, we need to figure out what room they can access and what are the limits of their freedom in these rooms…..most likely they will be limited to the public space room ,and multiple university rooms of their choice(there need to be a prompt for the guests to choose between the existing university rooms in the system).

As for interactions in the said rooms and the way the platform will handle that type of users…there is still a conflict in ideas…

****“Only comment on their own posts....+they will be required email if they want to keep the guest session? Or not keep the session at all and be terminated after 24 hours + just number the guests”

****”In that case why don't have a guest and a pre-student role? (Don't shoot me)”sorry frog….

This might be my final verdict on the guest logic(I hope)

Guests are restricted primarily to the public space and selected university rooms of their choice, which they can access in a read-only manner. A prompt allows guests to choose a limited number of university rooms to personalize their browsing experience. Around 3 to 5 maybe? Oh yeah.. there will be absolutely no saving f the session, once the browser tab is closed, everything the guest saw is gone….till their next visit

Guests cannot post, comment, vote, or interact in ways that affect platform content, credibility, or user ratings. This preserves the integrity of the reputation system while allowing potential users to understand the platform’s value before registering.

This of course means that there will be constant feedback for every move they make….

Examples:

> “Log in to ask a question”////“Create an account to vote”////“Join to follow this discussion”///so on so forth…this should be clean, right?

It will preserver the rights and actions the users with accounts have…..

By that logic, when the process of creating the university rooms is over-with, we need to create a prompt that gets activated when the “guest” button is clicked…to allow the visitor to select about 5 universities max to overview and add the public space room.

2- Info form

During the early stages of coding I made a checklist for each page to keep up with what each one needs to feel alive and interactive…this is the list for the info page:

** validate the age limit to 17 or more ✅

** maybe add the age max at some point (people born in 1000 can register apparently...)

** validate EMAIL format ✅

** the full name should have at least one space (my testing is the reason)

** handle form validation => form validation feedback

** before everything else/i had the bright idea to use choice.js (i need to download that too)

** update the form submission data ✅

** student gets one major ,prof gets multiple ✅

** the ‘other’ option in the university selection ✅

** figure the tag name and passage..

In the last note, so users know if they ae interacting with either a prof or a student…we figured that each username displayed will have a sort of tag or a badge for distinction

(prof=>professor /\/\/\ stdn=>student)

(prof=>professor /\/\/\ stdn=>student)

This is absolutely NOT editable by the user…fixed with the account. Stored once, displayed everywhere.

**in the radio choice of role, click student u get student, then click prof you get prof, then click student again…..u get both of them… this definitely needs to be fixed…(ps,I more or less made a decent fix for it)

Our extra move is to fight with choice.js and make it presentable the best we could yet in that sense…. Fix or improve the overall css for this form .

when the user cant find thier uni or something ,they choose other..simple enough right?

//except that we planned that each uni will have its room..right?

//see the mix up and the bug yet?

//well, A TOGGLE ..input toggle to fill in your uni..if it exists..the the something will derect you to that room...if not ..then something will create it…

All the info from this page so far is being saved to the local storage(front end test)

This will later be replaced by backend and API calls to save to an actual database

Something that keeps bugging me is the professor role…..right now any student can falsely choose a professor role and continue with their registration….this should be dealt with somehow….should we add an input of some kind of proof? Or trust the users (not gonna happen) honestly when it comes to this decision….to be frank…the only difference I notice so far would be the tag next to the name and the major list selection….am I missing something? this should be on the consideration table for future plans (might ask a senior dev or reddit opinion on the matter).

3-register

//password===confirm-password ✅

//that checkmark thing==>maybe give up on that?

This is for checking is the password and confirmed password are matching then a green checkmark will appear next to both…

This ,I changed with animations for the button….but even that doesn't seem to be correct for some reason….needs more reflecting on it.

//add a password strength indicator ✅

//add a username regex or something to actually have a username(english + brain are not doing thier work now ...) ✅

//=>bisaclly the username would have a number or a special character of some sort=>is this necessary though? Only if the user wants to…

//

//handle form validation =>form validation feedback

//save progress with local storage

//maybe add a loading spinner or subtle animation when the form is submitted.

//following the username function,CHANGE THE DAMN COLORS AND FONT STYLES TO SOMTHING MORE APPEALING ✅

//steal the colors from the login page later ok? ✅

About the button activation…change that so when the password input matches AND the username is there .

By simple logic, this page gives the user the freedom to choose the name they will be known for.

User name thus password, to keep their unique style are judged by a set of rules from length to what character is acceptable and what not….

Like all parts of project, this is still work in progress ….there is absolutely no way the passwords will be saved as they are (as we are testing in the local storage model) in the future backend and API, there will be hashing so the passwords will be one way encrypted for the security of user's data.

On another note….study the error management dude….lets not do alerts, give real feed back prompts and actual guidance ….lets not throw the users to a maze and leave them there with no clues of how to get out….

4-fin

Other than ensuring the data is passing correctly among the pages ….i don't see what we can add to this page…

It is a simple celebratory page, it has confetti, it has links to guide to where ever the user might possibly want to go, it even has a link behaving as a button to trigger the confetti……

I simply can't see what else could be added here….

5-log in

What might be one of my most productive decision about this page is that it shares almost everything with the register page.. but still each has its own functionality thus the similar things are coded once and shared in between .

I mentioned before I potential bug/error hazard when it comes to choice of login….my initial thought was simplify it and only login with username and password….considering the user base would be similar to me-this idea isnt very practical….other then the password being forgotten (which we will take measures for) I, for atleast once has forgotten the username of one of my accounts…..this cant stay so we need to make the login options (username|| registered email)&& password…..

//add input validation to back the required of html

//failed login feedback (when backend IS connected){i'm scared of the backend phase tbh :)}

//validate the username again (could be the same function as the register page) ✅

//vAliDatE the PassWoRd AgAin...

//be evil and add a login attempt limiter :)=>sadly this is not possible without backend but still...

//be a nice person and add a "Remember Me Checkbox"to store username in localStorage/coockie for convenience

//instead of only relaying on the username to login,i should add an email input aswell to avoid username forgetfulness

//show password toggle

//handle form validation feedback

Since there needs to be a password passing here….the backend will also implement the same hashing for the register page, but this time the API will only compare the hashed password…if matched to the email or username's provided password…that means login successful…..if not then its an error to be handled.

//steal the colors from the register page later ok?

And this is the point where I again remind me and you about the error handling feedback…..this definitely needs to be a thing, at the login stage…the only 2 errors that could happen are mistakes at either inputs….simple message : "Invalid credentials" should be enough to deal with this… again, lets try the best and avoid alerts for this.

And if all errors are avoided, then login successful thus user gets directed to dashboard.

6- request to reset password + reset password

as I mentioned before, users naturally forget their password ….this doesn't mean the account is lost, this simply means it has to be recovered via the email the user provided during their registration journey.

When the provided email is valid, the API will check its availability in the database…if there was the said email, the user will get a link with the reset password page to change their password and confirm it…once that is done (change password button is clicked) and the system accepts the process, the user will then be directed to the login page again to try and login with the new credentials . any error after that is more or less on the user…

As discussed on the register and login page, the same module of password hashing will be implemented ..this time to replace the existing password of the said account in the database.

There will also be error handling as failing in sending the reset email link, the updating of the new password, the new password cant be the same as old(this will be funny), the strength indicator of password the same as register…

There is also the functional side, as we still need to figure if this is the right set of steps or change it to simple OTP like code sent to the email to allow login this time only, or figure how to send the said email….this is still one of the mysteries of this project so far.

Also, the list I have for the request to reset password (I seem to not have one for the reset password page yet, will eventually be added):

//lets start simple by validating the email format (already have one that i can use in the index page)✅

//disable button until Email IS valid (i'm not doing all that just to get faulty email submitted)/✅

//add alerts of successor fail of sending reset emails???????why do i want this again? isnt the confirmation going to be in the email sending????

//actually implementing the sending of rest emails (although i think this is for the backend)

As for the rest password page, I think this should be enough as a starter:

-password strength indicator

-toggle to show/hide password typing

-deactivate button till password match confirmed password

-handle submission

My idea of error handling is if that email given doesn't exist in the database, then the feedback message should simply say "you sure about that 'email'?"

So far so forth, for the reset password page…the connection to it and where it leads next will be backend only……

Doesn't the otp code require expiration dates? That is logic as well and extra because it will be the extra step of account recovery over the email sending?

Maybe sticking with the page and backend handling it correctly is the best options for this project?

Wait….reset emails should expire as well…..they cant be valid for ever this will cause problems as well…. It should also be one time use….u cant keep resetting your password with the same email every time you feel like it……this also need to be studied and will be backend handled.

7-dashboard

Given that I have made 3 types of dashboard so far yet the only real progress I made is the dark looks of it…I'm not progressing much in what's supposed to be the main heart of the project…..that is spirit breaking but this is the reason of these plans….

//Live Counter Updates

//Recent Activity Feed

//Theme Toggle (Dark/Light Mode)

//Logout Confirmation

//Profile Preview on Hover

//Settings Panel Toggle:

//=>Instead of navigating away, let "Settings" open a slide-in panel with options like:Change theme/Update profile/Notification preferences

//Maybe add confetti to be activated on milestons????like 100 followers or something...

//Interactive Sidebar Navigation:

//=>Let the sidebar expand/collapse with a toggle button. Add smooth transitions and icon animations for flair

//Mini Modal for Profile Preview:

//=>Clicking "Profile" opens a modal with avatar, bio, and quick stats. (page on its own)

//Search Bar with Filter:

//=>Add a search bar that filters recent activity items as the user types.

Most of what I had in this list is the most simple nonsense there is,

What I have so far is the side bar would have access the directions links, light/dark mode toggle, …….in dashboard 2,we got interactive icons to access notifications, write posts, see followers…

Then the display of the logged in user's username…that is it….no actual good features I can name…at least, for now.

The plan is for the dashboard to have a filter to display the rooms the user joined and wants to participate in, thus they can post and interact clearly….

Frankly and simply, this page should work as a display a header with a welcome "username" +the tag either prof or stdn.

A sidebar with a home button to reset any filter done or any action…a rooms button to display the existing rooms(this might just be the user's joined in rooms or every other room

related to the said user -the subject rooms as all-), a profile button to display the profile of the user and based on the tag the profile for the student is slightly different from the professor profile., a logout button to lead to login page….and the light/dark mode button which I cant decide if it should stay as toggle or a button.

More or less , an about button that when clicked…it leads to a description of what this platform is – the core idea of the project-

A main where it will display a decent filter with all the rooms listed in it(could be a drop-down list of checkboxes for multiple choice) and the feed area where it will display the other users' posts in the similar rooms they chose.

To make your own specific post….2 choices, either go to the specific room related to that post where there will be a create post button, or a uniformed create post button and it will also have a filter to select the room related to the post(BOTH).

Each post will have the useful/useless voting system discussed in the core idea. There of course will be the ability to comment on that post, comments will have the 3 types of voting system, and there will be the ability to reply to comments (the replies will also be under the same voting system)

Users will get the choice to "save" a post and it will be stored in a special segment in the user profile.

Each comment and post will display time posted in and the username of the said post/comment/replay to comment…..

Op(owner of the original post) and the high rated users can give the third type of react which in calculating the rating of the user (specialized ==3 or 5 usefull) useless is -1,useful is 1 ….then we will figure a scale to put it /5 or similer idea.

Comments and posts can be edited or deleted by ONLY THEIR POSTER, no one else….

Comments and also posts can have photos in the as first base will be text and second base will be photos.

About the moderating…i(*we, sorry frog) have plenty of ideas…. if the comment or post gets enough useless votes…the said would be removed?….

Or should there be admins to moderate each and every activity? At the beginning of each year each room conducts an automatic poll to select 5 profs and 5 students of the highest ratings in the room? The highest chosen would be the admins of the said room till the end of the year.

THE MODS : like what reddit have…bot mods to detect some words or something and take action based on them.

Private rooms solve this problem because the creator of the room is responsible for it.

REPORTING: each comment and post would have “report to moderator” option ,so if an admin missed something, they can check it later if it was flagged by a user.

Users of private rooms can also report those private room…to us? This will keep everything under the watch and save the reputation (the non existing one) of the platform.

What I find most difficult in all this is how I might create the rooms and implement them correctly into the system ….. if we can find a clean why to code the rooms …I feel like the heavy part of the dashboard will be solved.

A cosmetic look for the sidebare, we could make make it like a toggle ….we get a profile picture of the user on the top left corner of the dashboard, click it and u get the dashboard… click it again and the toggle will hide it….this is extra step to be studied later on.

Why not devide it to layers? this might help clean things out…

Layer ONE**:** *Identity & context (simple)*

Welcome message:
“Welcome, username”

Tag/badge:
Student or Professor

Logout

Theme toggle

This layer **never changes**, regardless of rooms or posts.

Layer TWO**:** *Navigation*

Your sidebar idea is already correct. Don’t add more.

Sidebar core buttons (final set):

Home → resets filters

Rooms → shows room list(all the existing rooms that the user can join-the subject rooms that they can join-)

Profile → opens profile view

About → explains the platform(maybe just use the core idea in this doc?)

Settings (optional panel because i don’t know what it will possibly have)

Logout

Theme toggle (toggle is correct, not a button)

Everything else (notifications, followers, etc.)

Layer THREE: Rooms

I need to take away the idea that rooms are pages….rooms are logic ,permissions , tags and filters……

It could only be a json type of storage [name, type, id, members….]

Does that mean that I need to separate the dashboard per room? More to reflect on…

Public Room (auto-added)

Purpose

Global discussions

Cross-major / cross-university topics

Announcements, general help

Rules

Auto-joined for every registered user

Read/write access

Guests may **read only** (later decision)

This is your platform's "commons".

University Room (auto-added)

Purpose

University-specific discussions

Policies, exams, events, campus life

Rules

Auto-added based on selected university

Only users from the same university see it

Read/write access

This anchors identity and belonging.

Major Room(s) (auto-added)

Purpose

Core academic identity

Discipline-level problem solving

Rules

Students → 1 major room

Professors → multiple major rooms

Auto-added at registration

This matches the earlier logic perfectly. i hope

Subject Rooms (user-selected, joinable)

Purpose

Focused help on specific subjects

Avoid clutter in major rooms

Rules

Not auto-joined

Visible via **Rooms button**

User manually joins/leaves

Feed shows subject rooms only if joined

This is a **very good scalability decision** and **freedom choices.**

Private Rooms (profile-based, NOT dashboard)

Purpose

Study groups

Research teams

Project collaboration

Rules

Created from user profile

Invite-only

Not part of public dashboard discovery

Access controlled by owner

Keeping this **out of the dashboard** and **in the profile** is the best move for now

avoiding mistakes:

Making every room visible by default

Mixing private collaboration with public help

Forcing users into subjects they don't care about

gained:

clarity

scalability

clean permissions

low cognitive load

Dashboard shows:

Public room (always)

University room (always)

Major room(s) (always)

Joined subject rooms (optional)

Rooms button shows:

Joined rooms (active)

Available subject rooms (joinable)

Profile page handles:

Private rooms creation

Invitations

Membership management

This keeps dashboard = discovery + activity,
and profile = ownership + control.

8-profiles

A huge fog is on the profiles….should students and professors have the same profile?

Or should they have different functionalities?

What I know is the profiles each should have a *sidebar* with a button to go back to the dashboard, a button to reach the rooms (only the ones the user joined-this is the difference between the dashboard rooms button and the profile rooms button), create a private room button(when clicking this the user will have a prompt to chose the name of that room and a create room pass key and a link to the said generated room to privately share), a my courses/my resources button where the user will find posts or resources they saved …

A button to lead to the followers list -or following list-….(those the user can share their private rooms with and can have one on one private conversation)

A logout and delete buttons which are self explanatory….

An edit button to let the user edit their email, username, password ONLY ,users can't edit anyother information about themselves because it will be a sensitive case.

A main with a header to display the username, profile photo, the tag, and the total rating of the user…,

A card like set of information and counters(total posts, comments, useful vote, useless vote, specialized gained - with when clicking on either of any of the past mentions, will lead to where and when all of the mentioned happened -, and more or less another followers/following(one not both-the other will be displayed by the button- counter that when clicked will show the list of followers, and a counter for rooms maybe…just a counter for this).

Maybe the professors will get a special segment in the main of the profile to show their courses (subjects of majors they are joined in)

Something to add in the dashboard when thinking about it is a must is a search function….users should be able to search the rooms if they don't feel like filtering it out, or search other users by the username…..which when they do….the search result will display a set of matching users…and when clicking on one of the users account they only get the main segment of the searched user's profile as a result.

It is at this moment we step to our next ,making real solid plans of what we discussed in what was past …..to make the vision come true.

Lets start with the framework to use for this one

React.js **https://react.dev/learn**

I think I'll keep a simple plan of answering 6 core questions about the choice of this one particular framework :

1. Why React is used in the project

2. What problems React solves in this project

3. What React features will rely on

5. How existing pages map into React components

6. Frontend tooling around React

Should we start?

1. **Why React is used in the project:**

   the project as it is already is heavy-duty, we have multiple pages with constantly changing states, roles, UIs…..if we kept building as it is(as simple as it is) this can cause at the very least 1 if not to many problems….

   We will have to build everything manually , keep passing data through dom ….this will eventually cause duplicate logic(don't believe me? Check the registration +login scripts)

   We will eventually if not soon enough fear to touch the code if not get lost in it…

   The project is an interactive platform with multiple user roles, dynamic pages, and continuous state changes (authentication, room selection, filters, posts, profiles).

   React is used to manage this complexity by structuring the interface into reusable components that update automatically based on user state and actions. This reduces manual DOM manipulation, avoids duplicated logic, and keeps the interface predictable as the project grows.

   Rather than reacting to events manually, the platform's interface reacts to state changes, which makes development safer, more scalable, and easier to maintain.

   React, a UI library, is more flexible and doesn't enforce any particular architectural pattern. React focuses primarily on the view layer and leaves a lot of the choices on application design up to the developers.

2. What problems React solves in this project

the interface depends on:

- who the user is (guest / student / professor)
- what they selected (rooms, majors, subjects)
- where they are (dashboard, profile, post, form)
- what they're allowed to do

Without React:

- UI state is scattered across DOM elements
- toggles desync (you fix one thing, break another)
- bugs like *"both selects show up"* happen

React solves this by:

- keeping state in one place
- letting the UI reflect state instead of forcing it

Problem 2: Repeated logic across pages

Already have:

- username validation
- email validation
- password strength logic
- role-based UI differences

Right now:

- logic is duplicated
- changes require hunting multiple files
- bugs get fixed in one place but survive elsewhere

React solves this by:

- letting you reuse logic through components and hooks
- enforcing consistency automatically

Example:

- one PasswordField logic
- used in register, login, reset password
- same behavior everywhere

Problem 3: Role-based interfaces (this is BIG in BFUA)

The project is **not** a single-user experience.

The UI changes based on:

- guest vs registered user
- student vs professor

- rating level (specialized users)
- OP vs regular user

Without React:

- this becomes a nightmare of conditions and DOM rules

React solves this by:

- conditionally rendering components
- clearly expressing *who sees what*

Example logic becomes readable:

- Guests see:
    - public rooms
    - read-only posts
- Students see:
    - joined rooms
    - post + comment
- Professors see:
    - extra profile sections
    - course-related visibility

All without duplicating pages.

Problem 4: Growing UI without breaking old work

React helps **protect existing progress** by:

- isolating features into components
- letting the expand without touching unrelated parts

Example:

- adding private rooms won't break public rooms
- adding search won't break filters
- redesigning profile won't break dashboard logic

3. How the pages translate into React components

At the highest level, BFUA has **these main views**:

1. Welcome / Landing
2. Authentication flow
    - Info Form
    - Register
    - Login
    - Reset Password
3. Dashboard
4. Profile
5. Room view (posts + comments)

React's job is to:

- keep these separated
- let them share logic safely

Pages (route-level components)

These represent **where the user is**.

- Welcome
- InfoForm
- Register
- Login
- ResetPassword
- Dashboard
- Profile
- Room

Each page:

- owns layout

- pulls smaller components inside it
- does **not** contain heavy logic

simply, to adapt react to the project…..the CSS stays as is(minor fixes if needed will be done - maybe using tailwind -) Javascript will have to change but only in the sense of the syntax…logic that took the longest to achieve will just be the same, written differently.

As for the HTML, the body will be reserved, with only the small change such as class→ClassName and so on so forth….

4. what parts we will need

React is HUGE and has many parts libraries and features….we are learners, we don't need to use everything….so I'll map what we will potentially need.

The project relies on core React features such as functional components, state management (useState), side-effect handling (useEffect), and client-side routing to manage its interactive interface. Conditional rendering is used extensively to differentiate user roles, permissions, and visibility of features.

Shared application state such as authentication and theme preferences may later be managed using React Context, while form interactions are handled using controlled components to ensure validation consistency and predictable behavior. The project avoids unnecessary complexity by limiting React usage to features that directly support its functional requirements.

5. How existing pages map into React components

As I said before…each page in react will become a component.

We will have to refactor the logic and reconnect them as we should have done from the start…..

5.1 Welcome Page

Current state:

- static content
- navigation to register / login
- guest entry

React mapping:

- WelcomePage
    - HeroSection

    - ActionButtons

Very light logic. Perfect first page to convert.

5.2 Info Form Page

Current state:

- heavy form logic
- role switching (student/prof)
- major selection
- validation

React mapping:

- InfoFormPage
    - RoleSelector
    - StudentMajorSelect
    - ProfessorMajorSelect
    - UniversitySelect
    - FormActions

All current HTML becomes JSX.
JS becomes useState.

This page is an *excellent* React training ground., if we can pull this one of…the rest of the html+css will be a peace of cake.

5.3 Register Page

Current state:

- username rules
- password strength
- confirm password
- live feedback

React mapping:

- RegisterPage
    - UsernameInput
    - PasswordField
    - PasswordStrength
    - SubmitButton

Important win:
password logic reused later in **Reset Password**.

5.4 Login Page

Current state:

- username/email login
- password check
- error feedback
- remember me

React mapping:

- LoginPage
    - LoginForm
    - CredentialInput
    - PasswordField
    - LoginFeedback

Same structure as register. Same components. Less code duplication.

5.5 Reset Password Flow

Two pages, shared logic.

React mapping:

- RequestResetPage
    - EmailInput
    - SubmitButton
- ResetPasswordPage

    - PasswordField
    - PasswordStrength
    - ConfirmPassword

5.6 Dashboard Page

React mapping:

- DashboardPage
    - Sidebar
    - Header
    - RoomFilter
    - Feed

Inside Feed:

- PostCard
    - VoteControls
    - CommentList
        - Comment
            - Reply
            - VoteControls

This structure:

- matches the vision
- scales cleanly
- survives backend integration*hopefully *

Profile Pages (student / professor)

Same page. Different sections.

5.7 Profile Page

React mapping:

- ProfilePage
    - ProfileHeader

- StatsPanel
- ProfileSidebar
- SavedResources
- FollowersList
- PrivateRooms

Conditional rendering handles:

- professor-only courses
- student-only info
- edit permissions

No duplication.

Routing + protection

React Router controls:

- guest access
- user-only pages
- redirect after login

Examples:

- guest → cannot access all the rooms
- logged-in user → redirected away from login to dashboard

6. Frontend tooling around React

What tools do I need *around* React so the project stays stable, readable, and sane? And most importantly …keep me from …lets not mention that here…

What we will have as a plan…(long overdo)

- React (functional components-library-)
- Vite (build tool)
- React Router (navigation)
- Vanilla CSS (existing styles)
- Light JS helpers (validation, storage)
- Selective third-party UI libraries (only when justified)-ehem..choice.js..ehem-

frontend is built using React with a lightweight build tool to ensure fast development and minimal configuration overhead. Existing CSS styles are preserved and reused to maintain

visual continuity, while JavaScript logic is progressively refactored into React components. Third-party libraries are used sparingly and only when they solve specific interface problems, ensuring the project remains stable, readable, and easy to extend.

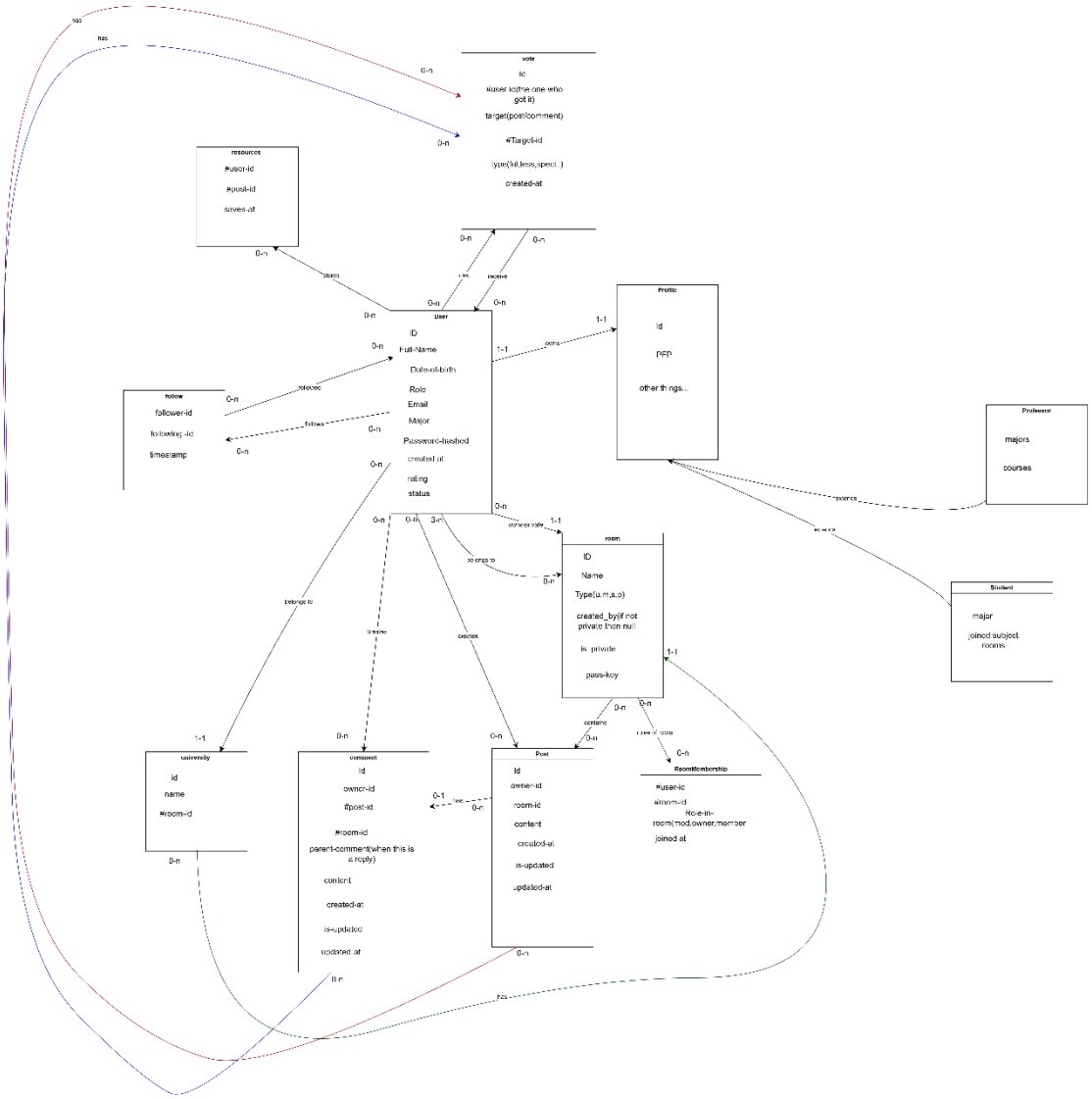
Btw, vite is the tool we will use to translate or migrate or whatever u want to name it ,the js to react native….it will make the process easier and faster…I hope…..

Backend (Node.js and what not)

From what we discussed earlier (me talking to my self in word)..the backend has a heavy duty with this project,

- Authentication & identity(all the security measures we talked about before)
- User roles & permissions(guest ,student, prof…and what comes with what..)
- Rooms & membership(who belongs where)
- Content storage(posta, comments, time stamps ,owner…)
- Voting & reputation(voting and rating and mods…)
- Recovery & lifecycle(email contact, password, account recovery)

backend is structured around a set of core entities that represent users, rooms, content, and interactions. Users are associated with a single university and may join multiple rooms of different types (public, university, major, subject, private). Content is organized through posts and threaded comments, with voting mechanisms used to calculate credibility and reputation. Relationships such as room membership, follows, and saved resources are modeled explicitly to support permissions, scalability, and future moderation features.



This is a primary MCD module to be updated in necessary later on….moving in the backend plans….i feel like learning a new language at this point will be a struggle to achieve and will hold the progress and the plans back….we will keep with JS…we will use Node.js(JavaScript +typescript)

As a frame work…why not Express.js, it should be simple enough to implement + MIDDLEWARE (this will come in handy for the mod bots)

API: **REST (clean and boring is good)**

The project needs:

- users
- rooms
- posts

- comments
- votes

REST is:

- easy to reason about
- easy to debug
- easy to document
- easy to test

Examples (mental model):

- /auth/login
- /users/:id
- /rooms/:id/posts
- /posts/:id/comments

Database **choice-tf?**

we use a relational database due to its structured and highly interconnected data model. PostgreSQL is selected as the core database technology for its strong support of relational integrity, constraints, and complex queries. A managed PostgreSQL platform may be used to simplify authentication, storage, and early development while preserving relational guarantees.

To simplify this…supabase would work great (we still need to learn it tho)..

While MongoDB is great for flexible data, the project is all about **integrity**. we need to ensure a student is *always* tied to a university and that votes *always* affect a specific user's credibility. PostgreSQL (Supabase) is designed to enforce these rules strictly, preventing bugs where a user's rating might get "out of sync".. ***this will be a pain to achieve…but so damn satisfying when it does.***